FlowBoard Guide

A Kanban REST API built to showcase software design & architecture

What this project is

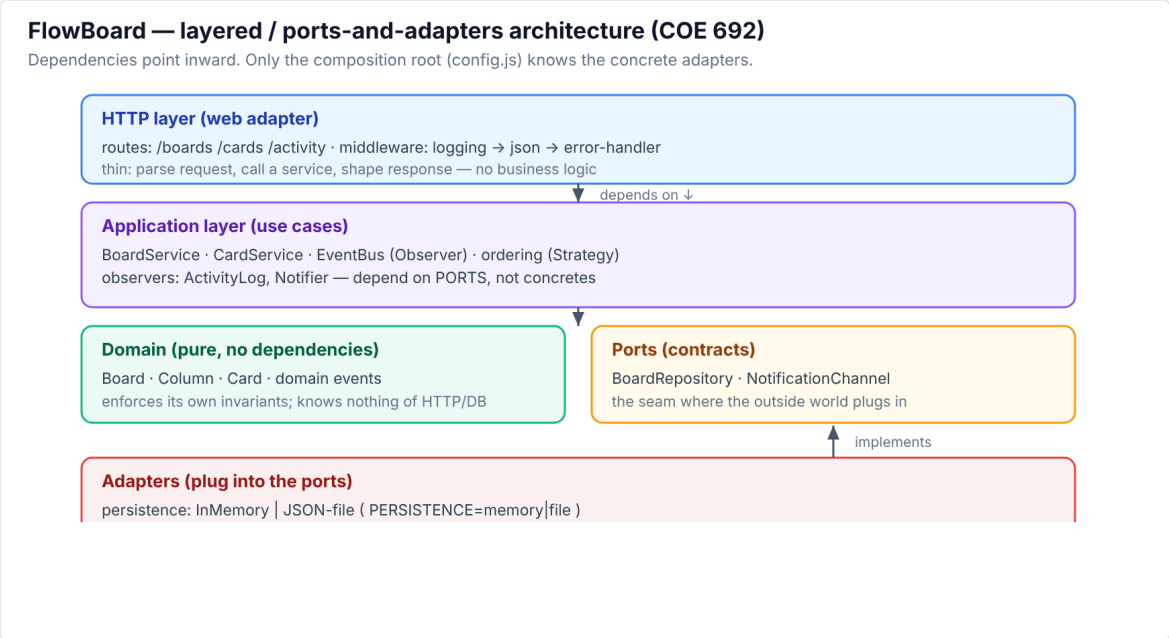
FlowBoard is a small Kanban board REST API — boards hold columns, columns hold cards, cards move, get assigned, and everything that happens shows up in an activity feed. The features are ordinary on purpose. The project is about **software design and architecture**, which is what COE 692 is actually about.

It's a **modular monolith**: one deployable, with a clean internal structure. The point isn't that it's big — it's that every structural decision is deliberate and written down:

- a **layered / ports-and-adapters** core, so business rules don't depend on HTTP or storage,
- an **event-driven** subsystem for the activity feed and notifications,
- a **plugin** seam so storage and notification channels swap by configuration,
- the decisions captured as **ADRs**, and the goals as **quality-attribute scenarios**.

It's deliberately *not* microservices — choosing the monolith, and being able to defend that choice, is part of the point (ADR-0001).

The architecture



FlowBoard's layered, ports-and-adapters structure

Dependencies point **inward**. The domain (Board, Column, Card) is pure and depends on nothing. The application layer (the services, the event bus, the strategies, the observers) depends only on the domain and on **ports** — abstract contracts like `BoardRepository`. The outside world (HTTP, storage, notifications) plugs in from the edge as **adapters**. The one file that knows which concrete adapters to use is `src/config.js`, the composition root.

Three styles, each with a job:

- **Layered / hexagonal** — isolates the rules so the web framework or the database can change without touching them.
- **Event-driven** — services publish domain events; the activity log and notifier react. The services don't know those features exist, so they were added without editing a service.
- **Plugin** — `PERSISTENCE=memory|file`, or adding a Slack channel, is configuration, not code.

The design patterns, and where to find them

Pattern	File	Role
Repository (port + adapters)	`ports/BoardRepository.js`, `adapters/persistence/*`	hides how boards are stored
Strategy	`application/ordering/orderingStrategies.js`	interchangeable card orderings
Observer / pub-sub	`application/EventBus.js`, `observers/*`	activity log + notifier react to events
Adapter	`adapters/notifications/*`	console & webhook channels behind one port
Factory	`domain/ids.js`	id creation isolated from entities
Middleware pipeline	`http/middleware/*`	logging → json → routes → error handler

How a request flows

`POST /boards/:id/columns/:col/cards {title, priority}`:

1. **Middleware** logs the request and parses JSON.
2. **Route** calls `cardService.createCard` — no logic of its own.
3. **CardService** loads the board via the repository *port*, asks the domain to create and place the card (which checks its own invariants), saves, and **publishes** `card.created`.
4. **Observers** react: the activity log records it, the notifier sends messages — the service has already returned.
5. The **error handler** maps any error type to a status (`DomainError` →400, `NotFoundError` →404).

Running it

```
npm install
npm test          # 4 tests, Node's built-in runner, no framework
npm run demo      # scripted tour of the API, no server needed
npm start         # API on http://localhost:3000
```

A few calls:

```
curl -X POST localhost:3000/boards -H 'content-type: application/json' -d
'{"name":"Sprint 1"}'
curl -X POST localhost:3000/boards/<id>/columns -H 'content-type:
application/json' -d '{"name":"To Do"}'
curl -X POST localhost:3000/boards/<id>/columns/<col>/cards -H 'content-type:
application/json' \
     -d '{"title":"Write ADRs","priority":"high"}'
curl "localhost:3000/boards/<id>/columns/<col>/cards?order=priority"
curl localhost:3000/activity
```

In a container (file persistence on a volume):

```
docker compose up --build
```

The documents to read, in order

1. **`docs/ARCHITECTURE.md`** — the C4 views, the three styles, and how a request flows.
2. **`docs/adr/`** — the three decisions: why a monolith (0001), pluggable persistence (0002), event-driven side-effects (0003). Each states the context, the decision, and the trade-off.
3. **`docs/quality-attributes.md`** — modifiability, testability, availability, performance, each as a concrete scenario, plus a trade-off table.

Putting it on GitHub

Create an empty `flowboard` repo (no README), then:

```
cd flowboard
git init && git add . && git commit -m "COE 692: Kanban API as a modular
monolith"
git branch -M main
git remote add origin https://github.com/munevvercoskun/flowboard.git
git push -u origin main
```

`node_modules/` and `data/` are git-ignored. GitHub renders the Mermaid diagrams in the markdown docs automatically, so the C4 and layer diagrams show up right on the repo page.

What to say about it

The honest, strong version: *"It's a Kanban API, but I built it to demonstrate architecture: a modular monolith with a ports-and-adapters core, so the business rules don't depend on Express or the database — I can swap in-memory storage for a file or Postgres by changing one line. Notifications and the activity feed are event-driven, so they're decoupled from the write path. I wrote the decisions up as ADRs and the quality goals as scenarios, and I chose a monolith on purpose — I can explain exactly when I'd split it into services and when that would just add cost."* That's the COE 692 skill set: not just building it, but justifying the shape.